Doc. no.:	P3907R0
Date:	2025-11-2
Audience:	LEWG
Reply-to:	Zhihao Yuan <zy@miator.net>

Waving more `::result_type` goodbye

- Waving more ::result_type goodbye
  - History
  - Discussion
    - Be consistent with… what?
    - What's the unfulfilled demand?
    - What can address them?
  - Proposal
    - Wording
  - References

History

Paper P0090[1] called for the deprecation and removal of all `result_type` member typedefs except for those in `<random>` backing in 2015, stated

> They're counterproductive, because many callable objects lack them. […], lambdas have always lacked these typedefs, and they are the most important function objects of all. […]
>
> These typedefs should be removed because they've become actively harmful to modern code.

When P0005R4[2] that contains the wording made into the pipeline in 2016, LEWG review recommended retaining `result_type` for only `std::function`, with feedback quoted "still actively using the feature."

All the `result_type` member typedefs deprecated in C++17 were removed since C++20. `std::reference_wrapper` supports incomplete types since then[3].

`std::move_only_function` in C++23 provides the `result_type` member typedef, so does `std::copyable_function` in the CD.

LWG issue 4373[4] asks for adding `result_type` to `std::function_ref`, stated

> It seems worthwhile to also provide it [...], as it is consistent with the other [standard library's polymorphic function]‡ wrappers and allows the user to easily extract the return type.

‡*Added content in later email exchanges.*

Discussion

Be consistent with... what?

The term "standard library's polymorphic function wrappers" is an interesting way of saying "`std::function` and `std::move_only_function`" – the only two polymorphic function wrappers in the standard library as of today in 2025.

There have been at least two papers calling for deprecating `std::function`, N4159[5] (2014) and P2721[6] (2024). The effort is to be revisited in the C++29 timeframe. In other words, the `result_type` member typedef in `std::move_only_function` is a sole instance "pattern" that replicates a to-be-deprecated candidate's to-be-deprecated member typedef.

Among the six `function_ref` implementations surveyed in P0792[7], ■ `llvm::function_ref` (https://github.com/llvm/llvm-project/blob/main/llvm/include/llvm/ADT/STLExtras.h), ■ `tl::function_ref` (https://github.com/TartanLlama/function_ref/blob/master/include/tl/function_ref.hpp), ■ `folly::FunctionRef` (https://github.com/facebook/folly/blob/main/folly/Function.h), ■ `gdb::function_view` (https://github.com/bminor/binutils-gdb/blob/master/gdbsupport/function-view.h), ■ `type_safe::function_ref` (https://github.com/foonathan/type_safe/blob/main/include/type_safe/reference.hpp), and

■ `absl::function_ref` (https://github.com/abseil/abseil-cpp/blob/master/absl/functional/function_ref.h), only `type_safe::function_ref` provides an typedef `return_type` that aliases the `R` template parameter. So, ironically, if we take "consistency" to mean "be consistent with the standard library," only one instance in the field offers a similar capability, and it asks for inconsistency.

An outreach survey revealed that `std::move_only_function` and `std::copyable_function` in the CD, with a clear intention to support multiple signatures in the future, fail to follow the existing practice in this field[8]. Neither the `cxx_function` library (https://github.com/potswa/cxx_function) nor `fu2::function` (function2) (https://naios.github.io/function2/) offers this kind of typedefs in the single-signature specializations. To look at it in the other way, if `std::move_only_function` and the `std::copyable_function` do support multiple signatures, the `result_type` member typedef inevitably becomes a resurrected *weak result type*.

The author sees consistency in staying on the rail that began with P0090[1:1]. That is, to not offer the `result_type` member typedef in callable types, other than those in `<random>`, as

- standard library types or closure types,
- built-in types or user-defined types,
- polymorphic or non-polymorphic,
- with a pre-determined return type or without,
- has a single signature or multiple signatures.

What's the unfulfilled demand?

"But `result_type` is useful!" Everything is useful including `first_argument`. But let's study some real code.

The author searched for `result_type` in a portion of his employer's codebase, found only its footage in Boost libraries and snippets related to random number generators, even though we have extensive use of `std::function`. An unproven theory would be: the point of using polymorphic callable wrappers in the first place is to lift compile-time polymorphism to runtime polymorphism, and a byproduct of this practice is that the full signature is available from the beginning.

The difference between `decltype` (or `invoke_result_t`) and `::result_type` is that the latter is a property of the callable entity as opposed to the call expression, so a trait to obtain that property would be a natural thought. And such traits do exist in the field, `llvm::function_traits` (https://github.com/llvm/llvm-project/blob/main/llvm/include/llvm/ADT/STLExtras.h) and `sqlite::utility::function_traits` (https://github.com/aminroosta/sqlite_modern_cpp/blob/master/hdr/sqlite_modern_cpp/utility/function_traits.h) from sqlite_modern_cpp are two (almost identical) examples.

Looking at their implementations and uses in each codebase, it is easy to conclude

1. Polymorphic callable wrappers are nothing special to their solutions. Their solutions work for most callables and can be extended to support all callable objects.
2. The `result_type` member typedef is nothing special to their solutions. It is not recognized.
3. The inquiries for return types appear neither more nor less frequently than others, such as parameter type at position I or the arity. The sqlite_modern_cpp repository does not even include any inquiry for a return type via `function_traits`.

In the meantime, a function type is neither an object type, therefore not a callable type. But analysing function types can be a building block for the aforementioned traits, and this building block is required in the implementations of modern polymorphic callable wrappers. The willwray/function_traits (https://github.com/willwray/function_traits) project captures this demand.

What can address them?

Putting these together, two traits should fulfill all known demand without loss of extensibility: `std::signature_traits` and `std::callable_traits`. The former takes a function type template argument in the form of `R(Args...)` *`cv_qual ref_qual`* `noexcept(`*`noex`*`)`, offers the following members:

```
  using unqualified = R(Args...);
  using return_t = R;
  template<size_t i>
    using param_t = Args...[i];
  static constexpr size_t arity = sizeof...(Args);
  static constexpr bool is_noexcept = noex;
  template<class U>
    using cv = U cv_qual;
  template<class U>
    using ref = U ref_qual;
```

The latter specializes for callable objects and function references, where deriving from the former serves as a typical implementation. For example,

```
  template<class T> requires is_class_v<T>
    struct callable_traits<T> : _callable_traits_impl<^^T::operator()> {};
  template<class F> requires is_function_v<F>
    struct callable_traits<F*> : signature_traits<F> {};
  template<class S>
    struct callable_traits<copyable_function<S>> : signature_traits<S> {};
  template<class T>
    struct callable_traits<reference_wrapper<T>> : signature_traits<T> {};
```

No doubt that a few members in `signature_traits` merely repeat information that you can obtain from reflection. Keep in mind that the "signature" in `signature_traits` is abstract – often no function of that function type exists. Its primary purpose is to set up a protocol for the rest of the program to know about callable objects. For the same reason, the traits do not support C-style variadic functions.

In this formulation, a default is provided for all user-defined callable types with a non-overloaded member function `operator()`, static or non-static, including standard library call wrappers and closure types. If users have additional needs, such as a shortcut or the type has a function template `operator()`, they can specialize `callable_traits`.

Here is a draft implementation WjqYe13Ks (https://compiler-explorer.com/z/WjqYe13Ks).

Proposal

Potential deprecation and addition are left for a future revision of the paper in the C++29 timeline. In the C++26 timeline, this paper proposes removing the `result_type` member typedef from `std::copyable_function`. Changing the new addition should be sufficient to discourage the use of `::result_type` without deprecating anything in the late stage.

Wording

The wording is relative to N5014.

Modify [func.wrap.copy.class] (https://eel.is/c++draft/func.wrap.copy.class) as follows:

```
namespace std {
  template<class R, class... ArgTypes>
  class copyable_function<R(ArgTypes...) cv ref noexcept(noex)> {
  public:
    using result_type = R;

    // 22.10.17.5.3, constructors, assignments, and destructor
```

References

1. P0090R0 *Removing `result_type`, etc.* https://wg21.link/p0090r0 (https://wg21.link/p0090r0) ↩ ↩
2. P0005R4 *Adopt `not_fn` from Library Fundamentals 2 for C++17.* https://wg21.link/p0005r4 (https://wg21.link/p0005r4) ↩
3. P0357R3 *`reference_wrapper` for incomplete types*. https://wg21.link/p0357r3 (https://wg21.link/p0357r3) ↩
4. LWG4373 *`function_ref` should provide `result_type`*. https://wg21.link/lwg4373 (https://wg21.link/lwg4373) ↩
5. N4159 *`std::function` and Beyond*. http://wg21.link/n4159 (http://wg21.link/n4159) ↩
6. P2721R0 *Deprecating `function`*. https://wg21.link/p2721r0 (https://wg21.link/p2721r0) ↩
7. P0792R14 *`function_ref`: a type-erased callable reference*. https://wg21.link/p0792r14 (https://wg21.link/p0792r14) ↩

8. P0288R1 *A polymorphic wrapper for all Callable objects (rev. 3)*. https://wg21.link/p0288r1 (https://wg21.link/p0288r1) ↩